

Lab 5 Report: Real Time Monte Carlo Localization in Mapped Environments with Sensor Data

Team 13 - TAMJAM

Tissany Chen
Jeryl Lewis
Ansis Anderson
Muktha Ramesh

Robotics: Science and Systems

April 10, 2026

1 Introduction and Motivation - Jeryl

For a robot to navigate and complete tasks it is essential that it accurately estimates its position and orientation (together known as pose) in its environment at any given time. This broad topic in robotics, known as localization, is complicated by noise in motion and sensor data. By utilizing known maps of the environment and real time LiDAR sensor data, our robot efficiently and robustly solves this problem of pose estimation using Monte Carlo Localization (MCL). This report explains the theory behind MCL and how it is implemented on our robotic system. We show that MCL allows for accurate pose estimation in real time. Furthermore, we explore the use of RTAB-Map, a visual-SLAM algorithm, to create the maps that MCL relies on.

2 MCL Approach - Jeryl

MCL is a localization method known for striking the balance between being accurate and robust to noise while having efficient real time performance. It works as a particle filter, representing the robot's pose by an adjustable number of particles. An initial set of particles get updated at each time step by a motion model that accounts for the robot's movement and noise in the system. The likelihood of each of these particles is then determined by the sensor model, based on real time LiDAR data. The set of particles gets resampled based

on this likelihood, which causes the distribution to converge over time. The estimate of the robot's pose is then given by the average pose estimate of the particles.

2.1 Motion Model - Jeryl

Given the previous pose of the particles $s_{t-1} = (x_{t-1}, y_{t-1}, \theta_{t-1})$, and the robot's odometry data $ds = (dx, dy, d\theta)$, the motion model gives the updated pose of the particles $s_t = (x_t, y_t, \theta_t)$.

To account for noise from the odometry data and other environmental factors we introduce:

$$\begin{aligned} dx_{noise} &\sim \mathcal{N}(0, \sigma_x^2) \\ dy_{noise} &\sim \mathcal{N}(0, \sigma_y^2) \\ d\theta_{noise} &\sim \mathcal{N}(0, \sigma_\theta^2) \end{aligned}$$

We choose to draw the noise from Gaussian distributions with spreads that will be empirically chosen based on experiment. We determined that the overall noise in motion comes from the sum of many small, independent factors, which are best modeled by Gaussian distributions. The same amount of noise might not affect x, y, θ , so it is desirable to choose the spread for each of these independently.

Then our updated poses are:

$$\begin{aligned} \theta_t &= \theta_{t-1} + d\theta + d\theta_{noise} \\ \begin{bmatrix} x_t \\ y_t \end{bmatrix} &= \begin{bmatrix} x_{t-1} \\ y_{t-1} \end{bmatrix} + R_{\theta_t} \begin{bmatrix} dx + dx_{noise} \\ dy + dy_{noise} \end{bmatrix} \end{aligned}$$

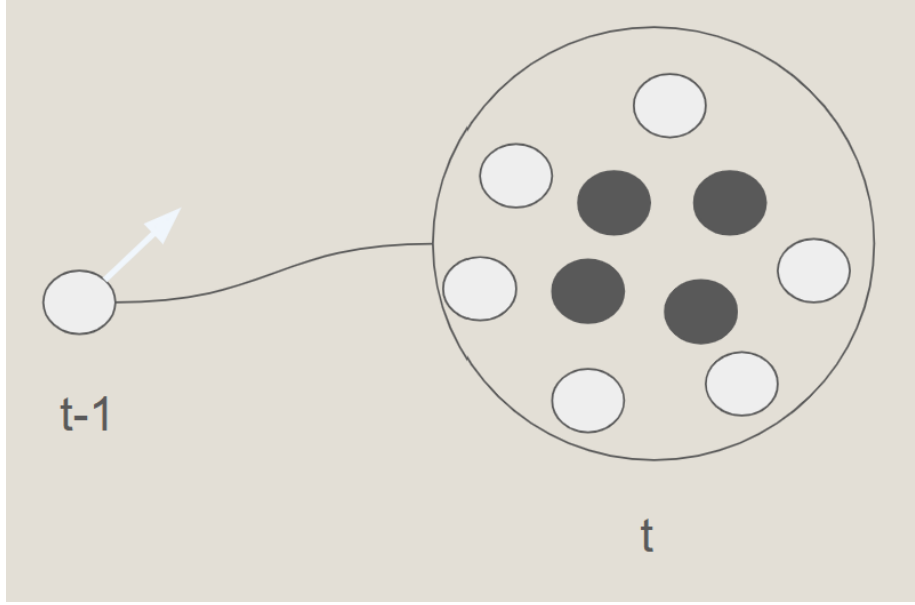


Figure 1: A representation of a MCL particle being updated over time according to the motion model. The more probable particles after the updated are shaded darker than the less probable particles. By adding Gaussian noise to the system, the pose of the particle after the motion model update is not deterministic.

2.2 Sensor Model - Jeryl

Given the current set of particle poses s_k , the n range measurements from the robot LiDAR data $z_k^{(1)}, \dots, z_k^{(n)}$, and the environment map m , the sensor model gives the likelihood that the LiDAR data measurement was taken from a particle's pose:

$$p(z_k | s_k, m) = p(z_k^{(1)}, \dots, z_k^{(n)} | s_k, m) = \prod_{i=1}^n p(z_k^{(i)} | s_k, m)$$

Here n , the ray count per particle is an empirically chosen parameter.

$p(z_k^{(i)} | s_k, m)$ is the weighted sum of 4 probabilities:

1. Probability of detecting a known obstacle in the map:

$$p_{hit}(z_k^{(i)} | s_k, m) = \begin{cases} \eta \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(z_k^{(i)} - d)^2}{2\sigma^2}\right) & \text{if } 0 \leq z_k^{(i)} \leq z_{max} \\ 0 & \text{otherwise} \end{cases}$$

where η is a normalization constant

2. Probability of a short measurement (detecting an unknown object):

$$p_{short}(z_k^{(i)}|s_k, m) = \frac{2}{d} \begin{cases} 1 - \frac{z_k^{(i)}}{d} & \text{if } 0 \leq z_k^{(i)} \leq d \text{ and } d \neq 0 \\ 0 & \text{otherwise} \end{cases}$$

3. Probability of a missed measurement:

$$p_{max}(z_k^{(i)}|s_k, m) = \begin{cases} \frac{1}{\epsilon} & \text{if } z_{max} - \epsilon \leq z_k^{(i)} \leq z_{max} \\ 0 & \text{otherwise} \end{cases}$$

4. Probability of a completely random measurement:

$$p_{rand}(z_k^{(i)}|s_k, m) = \begin{cases} \frac{1}{z_{max}} & \text{if } 0 \leq z_k^{(i)} \leq z_{max} \\ 0 & \text{otherwise} \end{cases}$$

We have: $\alpha_{hit}, \alpha_{short}, \alpha_{max}, \alpha_{rand}$:

$$\begin{aligned} p(z_k^{(i)}|s_k, m) = & \alpha_{hit} \cdot p_{hit}(z_k^{(i)}|s_k, m) + \alpha_{short} \cdot p_{short}(z_k^{(i)}|s_k, m) \\ & + \alpha_{max} \cdot p_{max}(z_k^{(i)}|s_k, m) + \alpha_{rand} \cdot p_{rand}(z_k^{(i)}|s_k, m) \end{aligned}$$

Where for our system: $\alpha_{hit} = 0.74$, $\alpha_{short} = 0.07$, $\alpha_{max} = 0.07$, $\alpha_{rand} = 0.12$, $\sigma = 0.4$ m, $z_{max} = 10$ m, and d is the distance calculated via ray-tracing from each particle pose s_k on the map m .

All together we have $p(z_k^{(i)}|s_k, m)$:

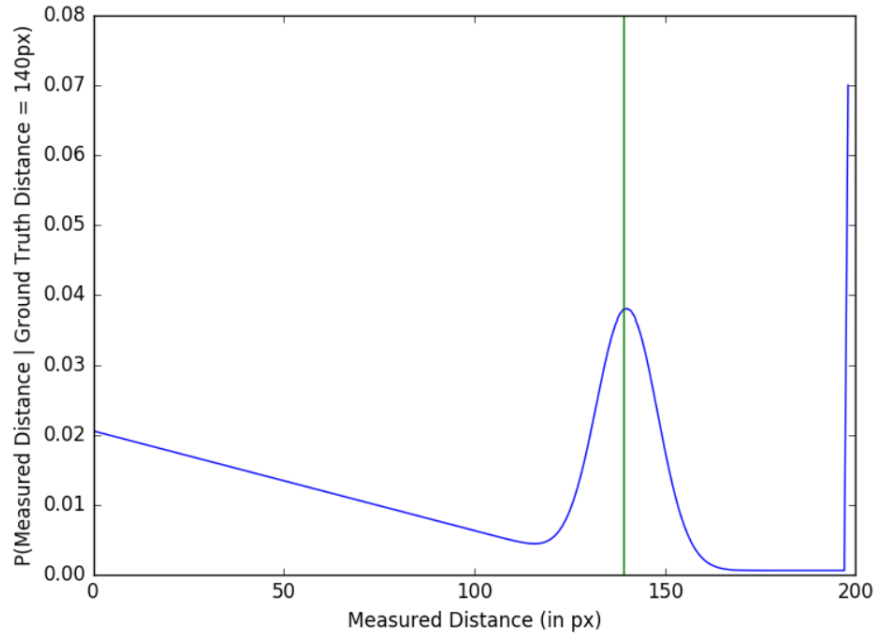


Figure 2: A graph of the sensor model probability distribution $p(z_k^{(i)} | s_k, m)$ as a function of distance in pixels. This graph gives a sense of what all the components that make up the probability distribution function look like when combined together.

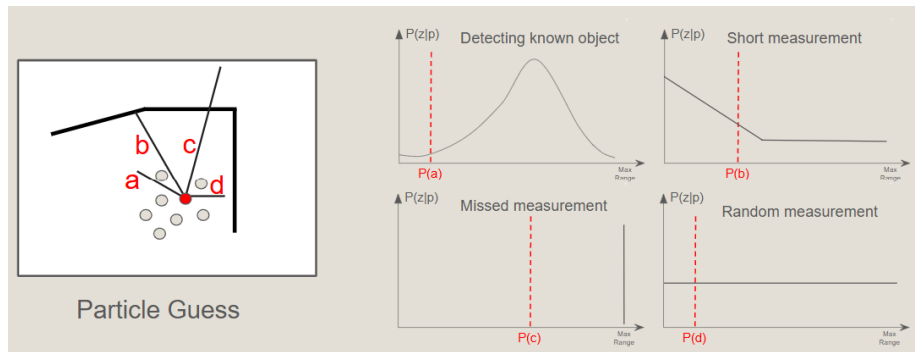


Figure 3: An illustrative example of how the sensor model works. For each particle range measurements are ray traced, and their probability is calculated

2.3 Particle Filter - Ansis

To finalize the implementation, the motion model is applied on every odometry update, while the sensor model is applied on LiDAR updates. After the weights of the particles are extracted, the particles are resampled based on these weights, and a slight amount of noise is added. This helps separate the particles, instead of keeping many copies of the same particle until the next motion model update. In practice, the sensor model does not run at the same speed as the LiDAR, however, since the motion model can provide a good estimate for short amounts of time, this does not have a major localization quality impact.

We designed two different particle initialization modes - one creates particles around a given point, allowing for fast convergence, the other creates the particles equally over the whole map to attempt global localization. In the final implementation, we removed the global initialization mode, since on the real robot, the hardware could not handle enough particles to converge reliably.

To extract the estimated position, we average the x and y positions of all the particles. Extracting the average angle is slightly trickier as it is a circular quantity, however, techniques for it are well documented. We convert all the angles to points on the unit circle, average these points and take the angle of the resulting point.

One thing to note is that if there is a minor difference in the weights of the particles (even less than 5%), this difference is repeatedly applied with every sensor model update, removing everything but the most likely particle very quickly. To combat this, we implemented a softening factor, and take the respective power root of all of the weights. This effectively multiplies the time it takes for other particles to disappear by the softening factor, smoothening out the distribution.

3 Evaluation in Simulation - Tissany

To set the adjustable parameters of our implementation of MCL and test its effectiveness we first turned to evaluation in simulation.

3.1 Metrics

The major inputs into our implementation of MCL include the particle count, the number of LiDAR rays to down-sample to, and the noise in our motion model. These inputs could affect MCL performance, which could be evaluated with metrics like cross track error, convergence time, and publishing rate.

1. **Cross Track Error:** Cross track error is defined as the Euclidean distance between the ground truth in the simulation and the converged localized position.

2. **Convergence Time:** Convergence time is defined as the time MCL takes to converge to the localized position after an initial pose is set.
3. **Publishing Rate:** Publishing rate is defined as the frequency at which our implementation of MCL updates its guess on a new position.

Minimizing cross track error improves the accuracy of the robot’s estimate of its position, minimizing convergence time allows the robot to have a localization more consistently while navigating, and maximizing publishing rate allows for better position estimates as it more closely matches the robot’s position in real time.

3.2 Evaluation Method

In every trial, rosbags are recorded where the robot is sent to an initial pose and then continues to drive down a hallway while localizing. A default parameter set of 200 particles, 100 rays, $\sigma_x = 0.06$, $\sigma_y = 0.02$, and $\sigma_\theta = 0.1$ was kept. Each new change in input parameter required three trials.

To evaluate the effect of particle count, particle count was varied while the rest of the default parameters were kept constant. Ray count and noise were varied similarly.

The evaluation script plotted localization errors, in terms of Euclidean distance between ground truth and localized poses. Figure 1 shows an example trial. Cross track error was determined by taking the mean of the last 20% of the recorded localization errors. Convergence time was determined by fitting an exponential decay function to the localization errors, and setting a default value of 0.1s when convergence is close to instantaneous. Publishing rate was determined by the frequency of the pose messages in the played back rosbags.

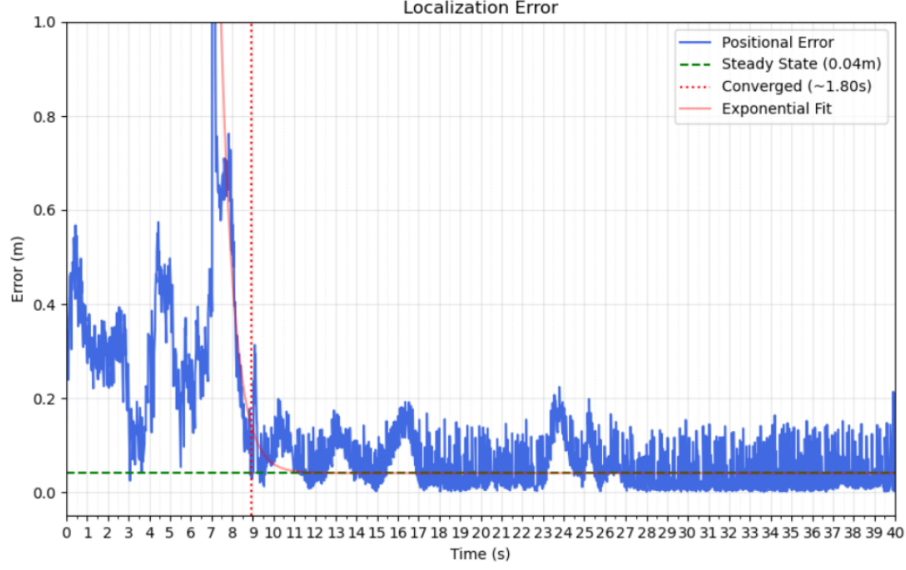


Figure 4: Localized position error is plotted as the robot drives in simulation for one trial. Cross track error is determined from the converged last 20% of the simulation time. Convergence time is determined after the peak in error, which corresponds to when the initial pose is set. This particular graph demonstrates how our implementation of MCL is able to quickly converge with satisfactory accuracy.

Limitations in this evaluation method included a limited number of cases tested for each relationship, a limited range of values tested, since it was difficult to know which values were interesting before testing began, and inconsistencies like different computer performance and slightly different initial poses while the rosbags were collected.

3.3 Results

Figures 5 through 9 depict the results of the simulation evaluation. Due to limitations in the evaluation method (large uncertainty, not enough data) , it made it difficult to definitively claim effects regarding motion model noise on our metrics.

From what we can see, increasing particle count decreases the time needed to converge and decreases cross track error, but also decreases publishing rate. Increasing ray count also decreases publishing rate, since more computation is needed for each iteration. Although it is hard to see an overall relationship for ray count and cross track error, the very high uncertainty at the low ray count of 10 is notable. At a ray count of 10, the high publishing rate can sometimes allow for lower cross track error, as the MCL updates more often, but local-

ization isn't as reliable. Also, there are particularly large uncertainties for the relationships between noise and publishing rate or cross track error.

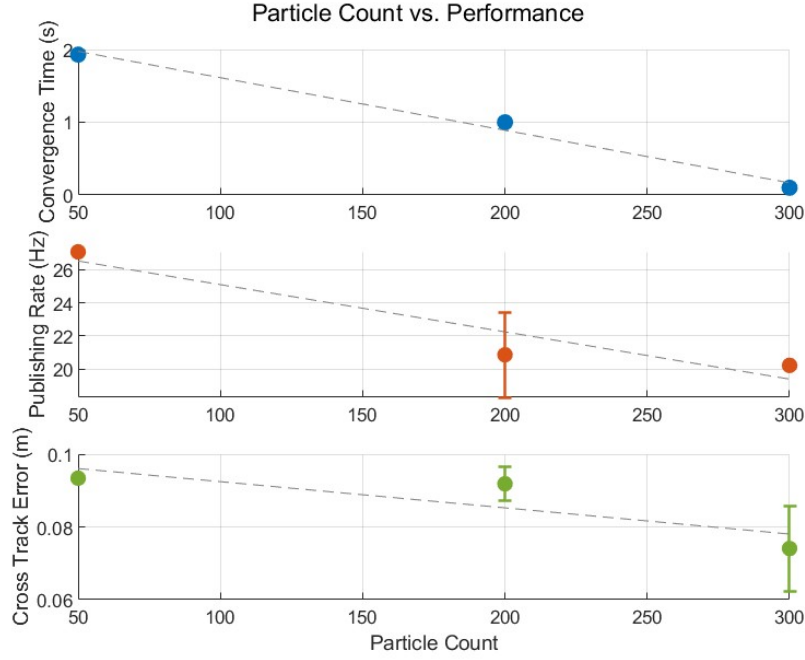


Figure 5: Graphs showing how varying particle count in simulation affects convergence time, publishing rate, and cross track error. We find that increasing particle count decreases convergence time, publishing rate, and cross track error. An increased particle count is associated with an increase in performance.

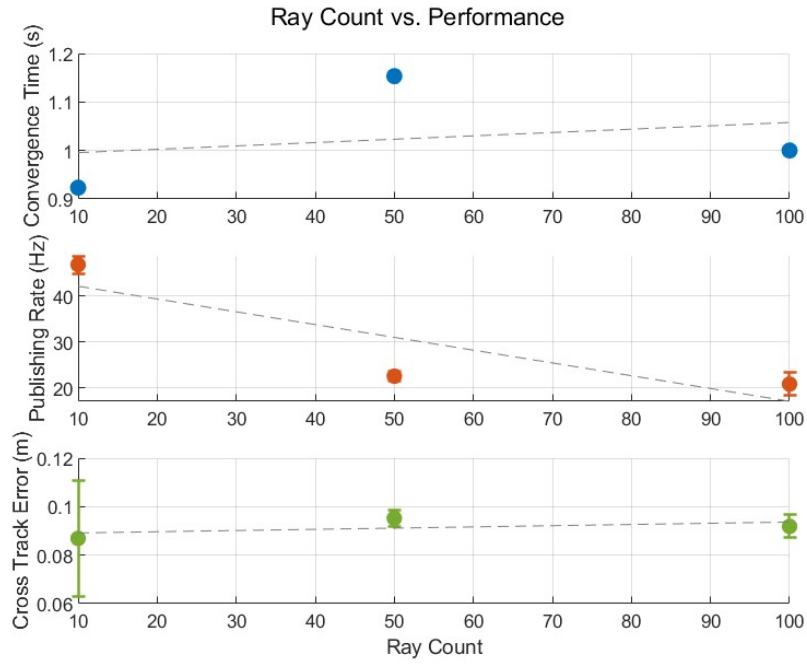


Figure 6: Graphs showing how varying ray counts affects convergence time, publishing rate, and cross track error. Increasing ray count decreases publishing rate. The effects on convergence time and cross track error is more uncertain. There is particularly high uncertainty for cross track error at very low ray counts, partly due to very high publishing rates lowering cross track error.

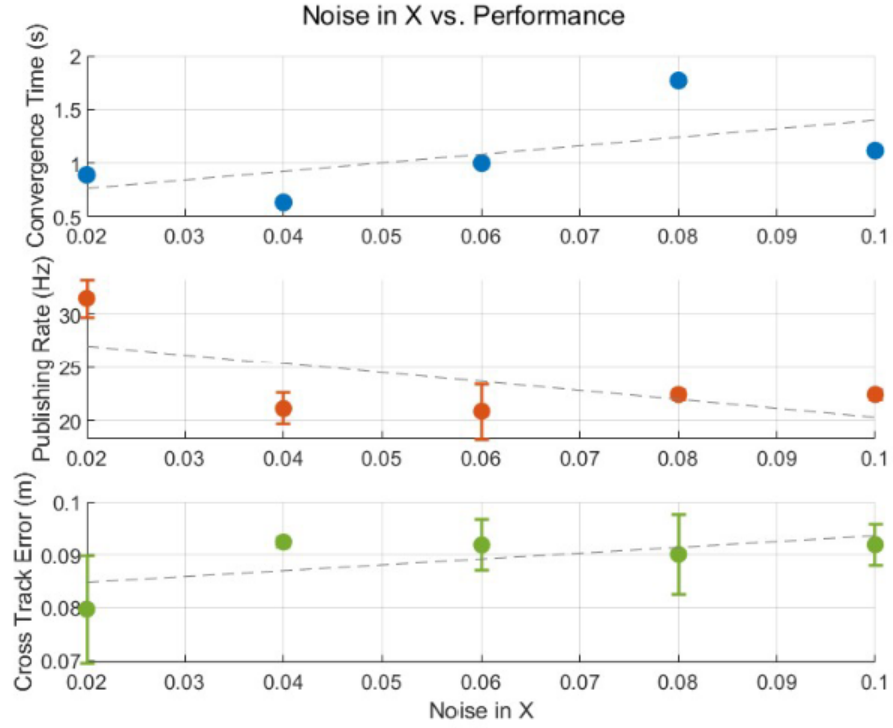


Figure 7: Graphs showing how varying noise in x affects convergence time, publishing rate, and cross track error. There are high uncertainties for the effect of noise on evaluated metrics. The limited range in tested input noise values also prevent clear relationships from being seen.

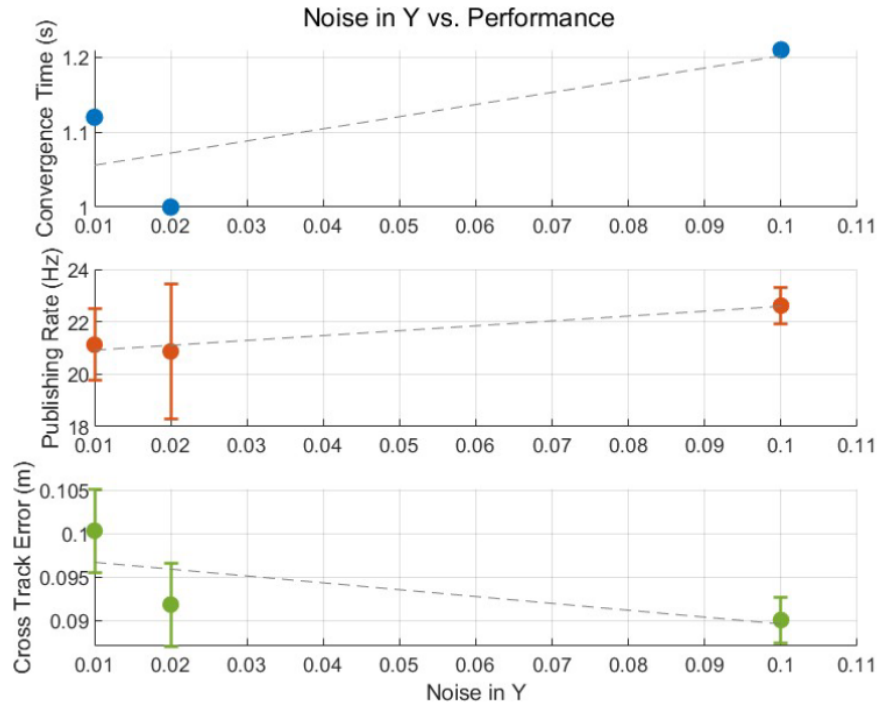


Figure 8: Graphs showing how varying noise in y affects convergence time, publishing rate, and cross track error. There are high uncertainties for the effect of noise on evaluated metrics. The limited range in tested input noise values also prevent clear relationships from being seen.

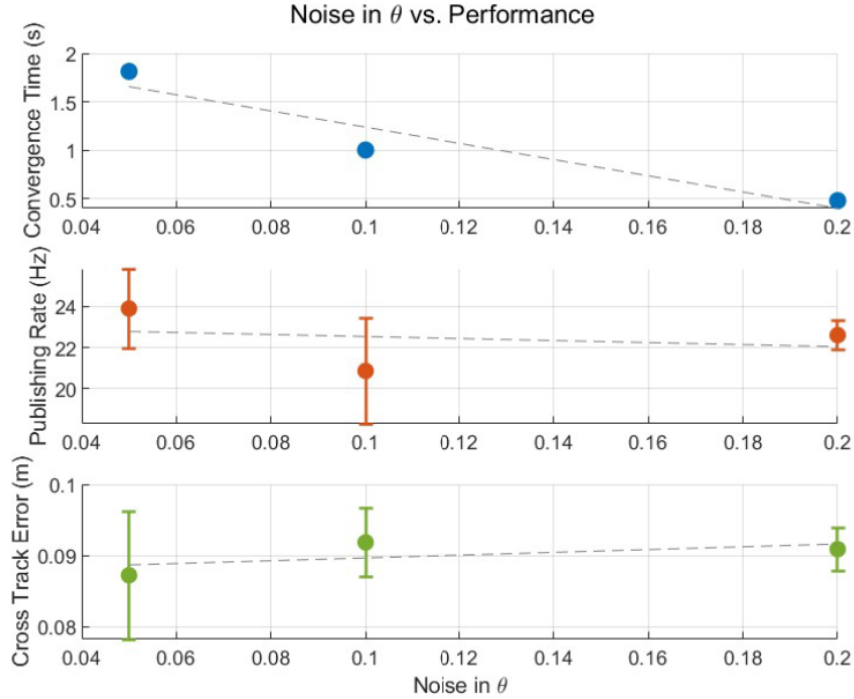


Figure 9: Graphs showing how varying noise in θ affects convergence time, publishing rate, and cross track error. There are high uncertainties for the effect of noise on evaluated metrics. The limited range in tested input noise values also prevent clear relationships from being seen.

4 Evaluation on Hardware - Muktha

4.1 Evaluation IRL

Next, we evaluated how well our particle filter performs in a real-world setting. This involves accounting for real-world challenges such as wheel slippage, sensor inaccuracies, and environmental variability. These factors can lead to more divergence between the predicted and actual robot states, making robust localization more difficult than in simulation.

First, we conducted an initial round of empirical testing to identify a set of parameters that allowed the robot to localize reasonably. This let us establish a baseline configuration for the motion model.

After determining a working baseline, we performed a more systematic evaluation by varying key parameters. Additionally, we tested different particle counts

and ray counts to see how they affected the following metrics.

4.2 Metrics

Unlike simulation, real-world evaluation does not provide access to ground-truth pose, preventing direct computation of localization error (e.g., cross-track error). The following metrics were used in evaluating the IRL localization:

1. **End Pose Accuracy:** Whether the final estimated pose of the robot is consistent with its true location at the end of a trajectory. Specifically, we check whether the MCL estimation of the final pose is within 2.5 meters of the true final pose.
2. **Particle Variance After Convergence:** The spread of the particle set after the filter has converged (the model has converged when the variance has reached less than 0.35 in our evaluation).
3. **Publishing Rate:** We measure the frequency at which pose estimates are produced by the particle filter. This reflects the computational efficiency of the system and its suitability for real-time deployment, as well as indirectly affects localization accuracy while moving as shown from the simulation.

In addition to these quantitative metrics, we qualitatively evaluated each trial by visualizing the particle distribution and estimated trajectory. This ensured that the filter exhibited expected behavior, such as convergence to the correct region and stability over time.

4.3 Evaluation Method

In order to ensure that the only variables at play were the parameters we were testing for, we ran MCL on rosbags collected from the robot. We utilized 4 unique test runs, and tested each parameter on each corridor 3 times (12 tests each).

For motion model parameter settings, our default settings were 0.1 for the x and y standard deviations, and 0.08 for our theta standard deviation. While keeping the rest of the parameters the same, we individually varied our x and y to be [0.01, 0.03, 0.1, 1], and theta to be in [0.02, 0.04, 0.08, 0.1]. After we got our best setting for x, y, and theta, we also tried the combination of the best standard deviation values for all three.

Finally, we also had attempted a variety of particle/ray counts. A few of these (# particle, # ray) settings included (50, 150), (100, 150), (100, 99), (200, 99), and (2000, 10).

Top 10 settings overall									
rank	particles	rays	sigma_x	sigma_y	sigma_theta	success_rate	var_after_convergence	publish_rate	
1	2000	10	1	0.1	0.08	0.917	didn't converge	66.924	
2	200	150	0.03	0.01	0.04	0.75	0.109	89.871	
3	50	150	0.03	0.01	0.04	0.667	0.112	90.097	
4	100	150	0.03	0.01	0.04	0.583	0.111	90.041	
5	2000	10	0.1	1	0.08	0.583	didn't converge	70.194	
6	100	99	0.03	0.01	0.04	0.5	0.121	90.095	
7	200	99	0.03	0.01	0.04	0.5	0.125	90.035	
8	2000	10	0.1	0.1	0.08	0.5	0.318	68.422	
9	2000	10	0.1	0.1	0.04	0.417	0.313	69.165	
10	2000	10	0.1	0.1	0.08	0.417	0.323	68.459	

Figure 10: Top 10 MCL parameter configurations ranked across four corridor environments. Each configuration is evaluated based on success rate, post-convergence variance, and publishing rate. Results highlight trade-offs between localization accuracy, robustness, and computational efficiency across environments. The two highlighted configurations correspond to the best-performing settings in terms of end-pose accuracy.

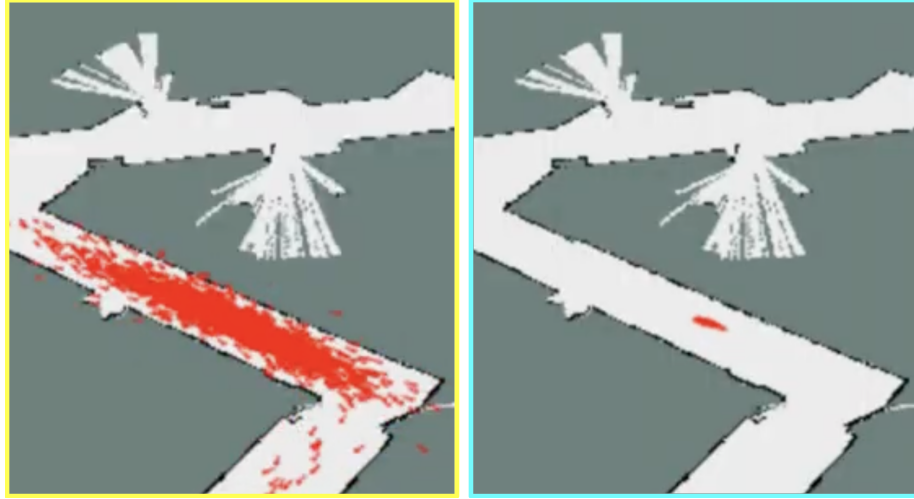


Figure 11: Comparison of particle filter behavior under different parameter settings. The left image corresponds to the yellow setting, the setting with the best end pose accuracy. It shows a highly dispersed particle distribution indicating poor pose convergence and high localization uncertainty in featureless areas such as a long hallway. The right image corresponds to the blue setting which had the second best end pose accuracy. It shows successful convergence, where particles collapse tightly around a consistent pose estimate even in a long hallway.

4.4 Analysis

Based on end pose accuracy alone, which seems to be the most important measure, it seems as though the yellow setting is the best. However, upon further inspection, the yellow setting struggled to do well in featureless areas such as long hallways. Due to the many particles and high uncertainties, this setting would lead to the robot being very uncertain about its position, which we can see as the localization didn't converge (variance never went below 0.35).

On the other hand, the blue setting did much better with localization in featureless sections. Once converged, the blue setting would output meaningful data throughout the long hallways. Although it did worse in end pose accuracy, it was much more useful in the middle of the robot trajectory, and did better in the other two metrics.

Top 10 settings for corridor1								
rank	particles	rays	sigma_x	sigma_y	sigma_theta	success_rate	var_after_convergence	publish_rate
1	50	150	0.03	0.01	0.04	1	0.101	90.093
2	200	150	0.03	0.01	0.04	1	0.104	89.898
3	100	150	0.03	0.01	0.04	1	0.104	90.052
4	2000	10	0.03	0.01	0.04	1	0.231	68.187
5	2000	10	0.1	0.05	0.08	1	0.337	68.112
6	2000	10	0.1	1	0.08	1	didn't converge	69.797
7	2000	10	0.1	0.1	0.08	1	didn't converge	68.275
8	2000	10	0.1	0.1	0.04	1	didn't converge	67.665
9	2000	10	1	0.1	0.08	1	didn't converge	67.376
10	2000	10	0.1	0.1	0.02	1	didn't converge	67.366
Top 10 settings for corridor2								
rank	particles	rays	sigma_x	sigma_y	sigma_theta	success_rate	var_after_convergence	publish_rate
1	100	99	0.03	0.01	0.04	1	0.104	90.204
2	200	99	0.03	0.01	0.04	1	0.108	90.159
3	100	150	0.03	0.01	0.04	0.667	0.099	90.149
4	200	150	0.03	0.01	0.04	0.667	0.103	89.941
5	50	150	0.03	0.01	0.04	0.667	0.132	90.125
6	2000	10	1	0.1	0.08	0.667	didn't converge	64.564
7	2000	10	0.1	0.1	0.08	0.333	0.334	66.448
8	2000	10	0.1	0.05	0.08	0.333	0.384	64.64
9	2000	10	0.1	1	0.08	0.333	n/a	67.778
10	50	99	0.03	0.01	0.04	0	0.099	90.223
Top 10 settings for corridor3								
rank	particles	rays	sigma_x	sigma_y	sigma_theta	success_rate	var_after_convergence	publish_rate
1	2000	10	0.1	0.01	0.08	1	0.433	67.778
2	2000	10	0.1	1	0.08	1	didn't converge	71.521
3	2000	10	1	0.1	0.08	1	didn't converge	66.55
4	2000	10	0.1	0.1	0.04	0.667	0.309	69.872
5	2000	10	0.1	0.1	0.08	0.667	0.31	67.022
6	2000	10	0.1	0.1	0.08	0.667	0.31	68.997
7	50	150	0.03	0.01	0.04	0.333	0.102	90.061
8	200	150	0.03	0.01	0.04	0.333	0.111	89.687
9	100	150	0.03	0.01	0.04	0.333	0.117	89.888
10	200	99	0.03	0.01	0.04	0.333	0.13	89.852
Top 10 settings for corridor4								
rank	particles	rays	sigma_x	sigma_y	sigma_theta	success_rate	var_after_convergence	publish_rate
1	200	150	0.03	0.01	0.04	1	0.12	89.959
2	2000	10	1	0.1	0.08	1	didn't converge	69.207
3	50	150	0.03	0.01	0.04	0.667	0.114	90.11
4	100	150	0.03	0.01	0.04	0.333	0.126	90.077
5	100	99	0.03	0.01	0.04	0.333	0.14	90.111
6	50	99	0.03	0.01	0.04	0.333	0.144	90.136
7	200	99	0.03	0.01	0.04	0	0.149	90.085
8	2000	10	0.03	0.01	0.04	0	0.291	69.745
9	2000	10	0.01	0.1	0.08	0	0.317	68.847
10	2000	10	0.1	0.01	0.08	0	0.326	68.157

Figure 12: Comparison of localization performance across different parameter settings in each of the 4 test cases. The results show that parameter choices significantly affect convergence stability, consistency, and robustness differently across environments.

In addition, when ranking by end pose accuracy and then variance, the blue setting actually did better in 3 out of the 4 test runs on average. Thus, we are confident in choosing the blue setting to put onto the robot.

5 Extension to SLAM - Ansis

Additionally, our team used the `ros rtabmap` package to capture a map of a lounge and part of the hallway in East Campus. This proved to be a challenging environment, mostly due to the tight hallway requiring several turns in each loop, heavily offsetting the dead reckoning odometry. The visual SLAM algorithm constructs a map at the same time as it localizes. This is possible due to incorporating images and a depth map from the ZED camera to identify repeating landmarks, and use that to adjust for the odometry dead reckoning errors. We found that driving extremely slowly, as well as adjusting the path to incorporate an easily repeatable loop, improved convergence speeds.

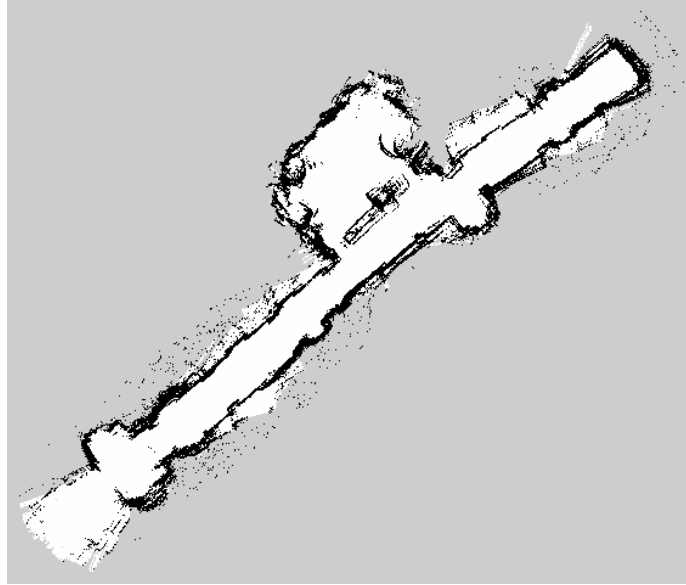


Figure 13: The final map captured after a 15-minute SLAM session. Slight curvature is still present in the hallway, which could be addressed by more loops; it does not have an impact on performance in this case.

To verify the captured map, we ran localization and visually compared the estimated position to the recorded video of the robot driving. As expected, the particle filter tracked the position very accurately, since the map was just captured and there weren't any unexpected obstacles or differences between the map and the real world. Due to this, we did not consider this map for evaluations, focusing on a robust evaluation on the more difficult Stata Center basement map.

6 Conclusion - Jeryl

Monte Carlo Localization (MCL) is an effective method to estimate the pose of our robot in a known, mapped environment. We have shown that MCL is able to operate in real time both in simulation and on the real hardware with satisfactory convergence rates and accuracy, even in sections of the map with less landmarks. We have additionally shown that we can generate maps to perform MCL with visual-SLAM algorithms. In the future, we can further improve our implementation of MCL by also experimenting more with the parameters of the sensor model.

7 Lessons Learned

7.1 Tissany's lessons learned

I learned that it is important to fully consider how much of the evaluation process to automate before starting. A lot of the uncertainty in the data could likely be mitigated if just one rosbag was collected so that computer performance and initial pose are more constant, but an entirely automated evaluation system could hide scenarios which benefited from seeing each localization error plot individually, like immediate convergence times that didn't work with the exponential decay model. I also learned that error bars can reveal a lot of detail about which relationships can be trusted, especially in cases with sparse data points.

7.2 Jeryl's lessons learned

Through this lab I learned that it can be really helpful to think about evaluation even before working on the implementation. By deciding the kinds of metrics you need beforehand and the types of testing you will be doing it becomes a lot faster to get and analyze the data. I also learned how important it is to redelegate tasks and keep a clear track of the progress of tasks.

7.3 Ansis' lessons learned

As I was focusing on the implementation side, I learned that it is easy to waste time overengineering features that don't actually make it into the final product. I wrote several features to help the robot recover from bad localization and return to a converged state; however, I ended up removing them since the filter updating more frequently and the robot never losing the localization in the first place was way more beneficial. Additionally, seemingly minor things can have a significant performance impact, for example, just publishing particles after every motion model update.

7.4 Muktha's lessons learned

On the technical side, I became even more comfortable with ROS, and especially with dealing with rosbags. I also learned a lot about localization. It was really interesting to see how much drift actually accumulated over time without localization, as well as figuring out how localization actually worked. Learning the quirks of MCL was also interesting, such as how dealing with long hallways (featureless areas) is actually pretty difficult for the robot, and how particle counts, ray counts, and different parameters in the sensor and motion models all contribute to whether the MCL will ultimately converge to the correct location. Even publishing rate ended up mattering, so optimizing that ended up helping accuracy, and tools like numpy were very useful here. When evaluating MCL, I also realized how memory intensive it was. Luckily for the robot, it could discard the particles immediately, but in evaluation I had to stop in the middle twice due to storage issues when holding 2000 particles published at 90hz. On the communication side, I think I learned how useful it is to spend the time we have together making a clear agenda for that meeting as well as the tasks for after the meeting was over. We do this most of the time, but I realized how useful it actually is in order to get things done.

Editor: Jeryl Lewis